Computational Mathematics for Learning and Data Analysis

Project Track 19 — Non-ML

QR Factorization and Limited-Memory Quasi-Newton Methods for Regularized Least Squares

Karol Jinet
k.jinet@studenti.unipi.it

Academic Year 2024–2025

Contents

1 Introduction

This project addresses the numerical solution of a regularized linear least squares problem. Given a data matrix $X \in \mathbb{R}^{m \times n}$ from the ML-CUP dataset, a random response vector $y \in \mathbb{R}^n$, and a regularization parameter $\lambda > 0$, the goal is to find the vector $w \in \mathbb{R}^m$ that minimizes

$$\min_w \left\| \hat{X} w - \hat{y} \right\|, \tag{1}$$

where the augmented matrix and vector are defined as

$$\hat{X} = \begin{bmatrix} X^T \\ \lambda I_m \end{bmatrix} \in \mathbb{R}^{(n+m) \times m}, \qquad \hat{y} = \begin{bmatrix} y \\ 0 \end{bmatrix} \in \mathbb{R}^{n+m}. \tag{2}$$

Expanding the squared norm, the problem is equivalent to minimizing

$$f(w) = \frac{1}{2} \left\| X^T w - y \right\|^2 + \frac{1}{2} \lambda^2 \left\| w \right\|^2, \tag{3}$$

which is a standard instance of Tikhonov regularization (ridge regression) [4]. The regularization term $\lambda^2 \|w\|^2$ penalizes large coefficient vectors, improving the numerical stability of the solution and controlling overfitting.

Two computational strategies are explored:

- **(A1)** A limited-memory quasi-Newton method (L-BFGS), which iteratively approximates the curvature of the objective function using only gradient information and a small number of stored vector pairs.
- **(A2)** A direct method based on thin QR factorization with Householder reflectors, in the compact variant where the orthogonal matrix Q is not formed explicitly, but applied implicitly through stored Householder vectors.

Both algorithms are implemented from scratch, without relying on any off-the-shelf numerical solvers. The first part of the report studies the iterative method (A1), while the second part derives the direct QR-based solver (A2).

2 Problem Properties

Before discussing the optimization algorithm, we analyze the mathematical properties of the objective function (3) that are relevant for convergence.

2.1 Gradient and Hessian

The objective (3) is a quadratic function with constant Hessian. Its gradient and Hessian are:

$$\nabla f(w) = X(X^T w - y) + \lambda^2 w, \tag{4}$$

$$H \coloneqq \nabla^2 f(w) = XX^T + \lambda^2 I_m. \tag{5}$$

2.2 Strong Convexity and Smoothness

Since $XX^T \succeq 0$ and $\lambda > 0$, the Hessian is symmetric positive definite:

$$v^T H v = \left\| X^T v \right\|^2 + \lambda^2 \left\| v \right\|^2 \geq \lambda^2 \left\| v \right\|^2 > 0, \quad \forall\, v \neq 0. \tag{6}$$

This implies that f is μ-strongly convex with parameter $\mu = \lambda_{\min}(H) = \lambda_m(XX^T) + \lambda^2$, where $\lambda_m(XX^T)$ denotes the smallest eigenvalue of XX^T. Since X is tall-thin ($m > n$), the matrix XX^T is rank-deficient with $\lambda_m(XX^T) = 0$ (see Lemma 2 in Appendix), hence $\mu = \lambda^2$.

The gradient is Lipschitz continuous with constant $L = \lambda_{\max}(H) = \lambda_1(XX^T) + \lambda^2$, since for all $w, w' \in \mathbb{R}^m$:

$$\left\|\nabla f(w) - \nabla f(w')\right\| = \left\|H(w - w')\right\| \leq \|H\| \left\|w - w'\right\| = L \left\|w - w'\right\|. \tag{7}$$

2.3 Condition Number

The condition number of the augmented matrix $\hat{X}$ is

$$\kappa(\hat{X}) = \frac{\sigma_{\max}(\hat{X})}{\sigma_{\min}(\hat{X})} = \sqrt{\frac{\lambda_1(XX^T) + \lambda^2}{\lambda^2}} = \frac{\sqrt{\lambda_1(XX^T) + \lambda^2}}{\lambda}, \tag{8}$$

which scales as $O(1/\lambda)$ for small λ. This means that weak regularization leads to ill-conditioned problems, directly affecting the convergence speed of iterative methods.

3 Limited-Memory BFGS (L-BFGS)

3.1 Quasi-Newton Methods: Background

Quasi-Newton methods are iterative algorithms for unconstrained optimization that approximate the Newton direction without explicitly computing the Hessian matrix [1]. At each iteration k, the search direction is computed as

$$p_k = -H_k \nabla f(w_k), \tag{9}$$

where $H_k \approx [\nabla^2 f(w_k)]^{-1}$ is an approximation to the inverse Hessian, built incrementally from gradient differences. The iterate is then updated as

$$w_{k+1} = w_k + \alpha_k p_k, \tag{10}$$

where $\alpha_k > 0$ is a step length determined by a line search procedure.

The BFGS update [3] maintains a dense $m \times m$ matrix H_k that is updated at each iteration via a rank-two formula using the pairs

$$s_k = w_{k+1} - w_k, \qquad y_k = \nabla f(w_{k+1}) - \nabla f(w_k). \tag{11}$$

While BFGS achieves superlinear convergence on strongly convex problems [1], storing and updating the full $m \times m$ matrix becomes prohibitive when the dimension m is large.

3.2 The L-BFGS Algorithm

The Limited-Memory BFGS (L-BFGS) method [2] addresses the memory limitation by storing only the most recent $\bar{m}$ pairs $\{s_i, y_i\}$, typically with $\bar{m} \in [3, 20]$. Rather than forming the full matrix H_k, the product $H_k \nabla f(w_k)$ is computed implicitly via a *two-loop recursion* (Algorithm 1).

3.2.1 Initial Scaling

At each iteration, the initial approximation $H_k^0 = \gamma_k I$ is set using [1, Eq. 9.6]:

$$\gamma_k = \frac{s_{k-1}^T y_{k-1}}{y_{k-1}^T y_{k-1}}. \tag{12}$$

This scaling attempts to estimate the magnitude of the true Hessian along the most recent search direction, ensuring that the search direction p_k is well-scaled and that the unit step length $\alpha_k = 1$ is accepted in most iterations.

3.2.2 Two-Loop Recursion

The product $r = H_k \nabla f_k$ is computed efficiently by the following procedure [1, Algorithm 9.1]:

Algorithm 1 L-BFGS Two-Loop Recursion [1, Algorithm 9.1]

Require: Gradient $q \leftarrow \nabla f_k$; stored pairs $\{s_i, y_i\}_{i=k-\bar{m}}^{k-1}$
Ensure: $r = H_k \nabla f_k$
for $i = k-1, k-2, \ldots, k-\bar{m}$ **do**
  $\rho_i \leftarrow 1/(y_i^T s_i)$
  $\alpha_i \leftarrow \rho_i \, s_i^T q$
  $q \leftarrow q - \alpha_i \, y_i$
end for
$r \leftarrow H_k^0 \, q$ ▷ $H_k^0 = \gamma_k I$ from Eq. (12)
for $i = k-\bar{m}, \ldots, k-1$ **do**
  $\beta \leftarrow \rho_i \, y_i^T r$
  $r \leftarrow r + s_i \, (\alpha_i - \beta)$
end for
return r

The cost of the two-loop recursion is $O(4\bar{m}\, m)$ multiplications, plus $O(m)$ for the scaling $H_k^0 q$. This is a significant improvement over the $O(m^2)$ cost of full BFGS.

3.2.3 Curvature Safeguard

The BFGS update requires that the curvature condition $y_k^T s_k > 0$ holds. When this condition is satisfied, the pair (s_k, y_k) is added to the memory; otherwise, it is discarded. If the curvature becomes excessively small ($y_k^T s_k \leq 10^{-16}$), the stored memory is cleared entirely and the algorithm restarts from a gradient descent step, following [2].

3.3 Line Search Strategies

3.3.1 Strong Wolfe Conditions

The standard approach for selecting the step length α_k in quasi-Newton methods is a line search satisfying the *strong Wolfe conditions* [1, Eq. 3.7]:

$$f(w_k + \alpha_k p_k) \leq f(w_k) + c_1 \, \alpha_k \, \nabla f_k^T p_k, \tag{13}$$

$$\left| \nabla f(w_k + \alpha_k p_k)^T p_k \right| \leq c_2 \, \left| \nabla f_k^T p_k \right|, \tag{14}$$

with $0 < c_1 < c_2 < 1$. Following standard practice for quasi-Newton methods [1], we set $c_1 = 10^{-4}$ and $c_2 = 0.9$.

Condition (13) (Armijo) ensures sufficient decrease in the objective value, while (14) prevents the step from being too short by requiring that the directional derivative at the new point is sufficiently small.

Our implementation uses a bracketing-and-zoom procedure with cubic interpolation, following the algorithmic framework described in [1, Section 3.5].

3.3.2 Exact Line Search for Quadratic Objectives

Since the objective function (3) is quadratic, the restriction of f along any direction p is a parabola:

$$\varphi(\alpha) = f(w + \alpha\, p) = f(w) + \alpha\, \nabla f(w)^T p + \frac{\alpha^2}{2}\, p^T H p. \tag{15}$$

The second derivative $\varphi''(\alpha) = p^T H p$ is constant and positive (since $H \succ 0$), so φ has a unique global minimum at

$$\alpha_{\min} = -\frac{\nabla f(w)^T p}{p^T H p} = -\frac{\nabla f(w)^T p}{\|X^T p\|^2 + \lambda^2 \|p\|^2}. \tag{16}$$

This is a generalization of [1, Eq. 3.25] to an arbitrary descent direction p. The computational cost is dominated by a single matrix-vector product $X^T p$, requiring $O(mn)$ operations — the same cost as a gradient evaluation, and considerably cheaper than the multiple function and gradient evaluations typically needed by the Wolfe line search.

For quadratic problems, the exact line search is theoretically preferable because it minimizes f exactly along each search direction, fully exploiting the parabolic structure of the objective. As noted by [2] and confirmed by our experiments (Section 4.1), this leads to faster convergence and higher final accuracy.

3.4 Complete Algorithm

The complete L-BFGS procedure is summarized in Algorithm 2.

Algorithm 2 L-BFGS [1, Algorithm 9.2]

Require: Starting point w_0; memory size $\bar{m} > 0$; tolerance $\varepsilon > 0$

$k \leftarrow 0$; compute ∇f_0; set `stop_tol` $\leftarrow \varepsilon\, \|\nabla f_0\|$
while $\|\nabla f_k\| >$ `stop_tol` **do**
  Choose $H_k^0 = \gamma_k I$ ▷ Eq. (12)
  Compute $p_k \leftarrow -H_k \nabla f_k$ ▷ Algorithm 1
  if $\nabla f_k^T p_k \geq 0$ **then** ▷ Safeguard: fallback to steepest descent
    $p_k \leftarrow -\nabla f_k$
  end if
  Compute α_k via exact line search (Eq. (16)) or Wolfe conditions
  $w_{k+1} \leftarrow w_k + \alpha_k\, p_k$
  $s_k \leftarrow w_{k+1} - w_k$; $y_k \leftarrow \nabla f_{k+1} - \nabla f_k$
  if $y_k^T s_k > 10^{-10}$ **then**
    Store (s_k, y_k); discard oldest pair if #stored $> \bar{m}$
  else if $y_k^T s_k \leq 10^{-16}$ **then**
    Clear all stored pairs ▷ Curvature reset
  end if
  $k \leftarrow k + 1$
end while
return w_k

3.5 Stopping Criterion

The primary stopping criterion uses a *relative* tolerance on the gradient norm:

$$\|\nabla f(w_k)\| \leq \varepsilon \cdot \|\nabla f(w_0)\|\,, \tag{17}$$

where $\varepsilon > 0$ is a user-specified tolerance and w_0 is the starting point. This criterion is scale-invariant: it does not depend on the absolute magnitude of the gradient, which can vary significantly across different values of λ.

As a secondary criterion, the algorithm detects *numerical stagnation*: if the relative change in function value $|f_k - f_{k-1}|/|f_k|$ drops below machine epsilon while the step length α_k is near zero, the iteration is terminated.

3.6 Convergence Analysis

3.6.1 Global Linear Convergence

The convergence of L-BFGS on the problem at hand follows from [2, Theorem 1]. The key assumptions are:

1. $f \in C^2$ (satisfied: f is quadratic);
2. the level sets $\mathcal{L} = \{w : f(w) \le f(w_0)\}$ are convex (satisfied: f is strongly convex);
3. $\exists\, M_1, M_2 > 0$ such that $M_1 \|v\|^2 \le v^T \nabla^2 f(w)\, v \le M_2 \|v\|^2$ for all $v \in \mathbb{R}^m$ and $w \in \mathcal{L}$ (satisfied: H is constant with $M_1 = \lambda^2$ and $M_2 = \lambda_1(XX^T) + \lambda^2$).

Under these conditions, the iterates converge linearly:

$$f(w_k) - f(w^*) \le r^k \, (f(w_0) - f(w^*)), \quad r \in [0, 1). \tag{18}$$

3.6.2 Superlinear Convergence

For the full BFGS method (i.e., when $\bar{m}$ is large enough to store the entire history), [1, Theorem 6.6] guarantees superlinear convergence on strongly convex functions.

A non-asymptotic analysis by Rodomanov and Nesterov [7] predicts that superlinear convergence begins after approximately $K_0^{\text{BFGS}} = 4m \ln \kappa$ iterations, where $\kappa = L/\mu$ is the condition number of the Hessian. Jin and Mokhtari [8] provide a local bound:

$$\frac{\|w_k - w^*\|_H}{\|w_0 - w^*\|_H} \le \left(\frac{1}{k}\right)^{k/2}, \tag{19}$$

where $\|v\|_H = \sqrt{v^T H v}$ is the Hessian-induced norm. This implies that achieving accuracy ε requires only $k = O\Big(\frac{\log(1/\varepsilon)}{\log\log(1/\varepsilon)}\Big)$ iterations, which is significantly better than the $O(\log(1/\varepsilon))$ bound for linear convergence.

Remark 1. *For L-BFGS with finite memory $\bar{m}$, the theoretical guarantee is only linear convergence. However, in practice, superlinear behavior is often observed for well-conditioned problems, as the limited-memory approximation captures sufficient curvature information when $\bar{m}$ is large enough relative to the problem dimension.*

3.6.3 Computational Complexity

Each iteration of L-BFGS requires:

- One gradient evaluation: $O(mn)$ (matrix-vector product $X(X^T w - y)$);
- Two-loop recursion: $O(4\bar{m}\, m)$;
- Line search: $O(mn)$ for exact (one product $X^T p$), or $O(k_{LS}\, mn)$ for Wolfe (where k_{LS} is the number of line search evaluations).

The total cost per iteration is $O(mn + \bar{m}\, m)$, and the memory requirement is $O(\bar{m}\, m)$ for storing the vector pairs — a drastic improvement over the $O(m^2)$ storage of full BFGS.

4 L-BFGS Experimental Results

All experiments use a matrix $X \in \mathbb{R}^{m \times n}$ generated randomly with i.i.d. standard normal entries (as a proxy for the ML-CUP dataset), with default dimensions $m = 600$ and $n = 12$. The response vector $y \in \mathbb{R}^n$ is drawn from a standard normal distribution with a fixed random seed.

Unless stated otherwise, the tolerance is $\varepsilon = 10^{-14}$ (relative), the memory size is $\bar{m} = 10$, and the baseline solution w^* is computed via the normal equations using `numpy.linalg.solve`.

4.1 Exact Line Search vs. Strong Wolfe Line Search

We compare the two line search strategies on the same problem instance with $\lambda = 0.5$.

Table 1: Comparison of line search strategies ($m = 600$, $n = 12$, $\lambda = 0.5$, $\varepsilon = 10^{-14}$ relative).

Line Search	**Iterations**	$\|w - w^*\|$	$\|f^* - f\|$	**Time (s)**
Exact	16	6.4×10^{-14}	4.3×10^{-19}	0.002
Wolfe	16	6.0×10^{-11}	$< 10^{-19}$	0.002

Both strategies converge in the same number of iterations, but the exact line search achieves approximately three orders of magnitude higher accuracy on the weight vector (10^{-14} vs. 10^{-11}). This is because the exact line search optimizes each step perfectly along the parabolic profile, while the Wolfe conditions allow a range of acceptable step lengths. The Wolfe variant stagnates earlier with a gradient norm of $\sim 4 \times 10^{-8}$, while the exact variant continues to decrease until $\sim 5 \times 10^{-13}$.

4.2 Effect of the Memory Parameter $\bar{m}$

We vary $\bar{m} \in \{3, 5, 10, 20, 40\}$ using exact line search, relative tolerance $\varepsilon = 10^{-12}$, and $\lambda = 0.5$.

For well-conditioned problems (moderate λ), the effect of $\bar{m}$ is minimal: all configurations converge within approximately the same number of iterations. This is consistent with the observation by Nocedal and Wright [1, Section 9.1] that modest values of $\bar{m}$ between 3 and 20 often produce satisfactory results.

For ill-conditioned problems (small λ), larger memory sizes provide a more accurate approximation of the curvature, leading to faster convergence.

4.3 Effect of the Regularization Parameter λ

We study convergence across several regularization regimes with $\lambda \in \{10^{-4}, 10^{-2}, 1, 10^2, 10^4\}$, using exact line search and $\bar{m} = 10$.

As λ increases, the condition number $\kappa(\hat{X})$ decreases (cf. Eq. (8)), leading to faster convergence. For $\lambda = 10^4$, the Hessian is nearly $\lambda^2 I$ and the algorithm converges in fewer than 10 iterations. For $\lambda = 10^{-4}$, the condition number exceeds 10^5, and convergence requires significantly more iterations.

This behavior empirically validates the theoretical dependence of the convergence rate on the condition number κ, as predicted by [7] and [8].

4.4 Relative Error Trajectory

To monitor convergence quality, we track the relative error

$$e_k = \frac{\|w_k - w^*\|}{\|w^*\|} \tag{20}$$

across iterations for different values of λ. For well-conditioned problems ($\lambda \geq 1$), the relative error decays smoothly to machine precision. For ill-conditioned problems ($\lambda \leq 10^{-2}$), the decay is slower and may exhibit non-monotonic behavior, reflecting the difficulty of accumulating accurate curvature information in flat regions of the objective landscape.

5 QR Factorization

We now turn from the optimization point of view to the direct linear algebra point of view. The same regularized least squares problem can be solved without generating an iterative sequence of approximations: one factorizes the augmented matrix

$$A \coloneqq \hat{X} = \begin{bmatrix} X^T \\ \lambda I_m \end{bmatrix} \in \mathbb{R}^{(n+m)\times m}, \qquad b \coloneqq \hat{y} = \begin{bmatrix} y \\ 0 \end{bmatrix} \in \mathbb{R}^{n+m}, \tag{21}$$

and then solves the resulting triangular problem. Since $\lambda > 0$, Lemma 1 shows that A has full column rank. Therefore the least squares problem

$$\min_{w\in\mathbb{R}^m} \|Aw - b\| \tag{22}$$

has a unique minimizer.

5.1 Thin QR Factorization

For a full-column-rank matrix $A \in \mathbb{R}^{p\times m}$, with $p = n + m \geq m$, the thin QR factorization is

$$A = Q_1 R, \qquad Q_1 \in \mathbb{R}^{p\times m}, \quad Q_1^T Q_1 = I_m, \quad R \in \mathbb{R}^{m\times m}, \tag{23}$$

where R is upper triangular and nonsingular. If $Q = [Q_1\ Q_2]$ is an orthogonal completion of Q_1, then

$$Q^T A = \begin{bmatrix} R \\ 0 \end{bmatrix}, \qquad Q^T b = \begin{bmatrix} c \\ d \end{bmatrix}, \quad c \in \mathbb{R}^m,\ d \in \mathbb{R}^{p-m}. \tag{24}$$

Orthogonal transformations preserve the Euclidean norm, hence

$$\|Aw - b\|^2 = \left\|Q^T(Aw - b)\right\|^2 = \|Rw - c\|^2 + \|d\|^2 . \tag{25}$$

The term $\|d\|^2$ is independent of w, while R is nonsingular. Therefore the least squares minimizer is obtained from the triangular system

$$Rw = c = Q_1^T b. \tag{26}$$

This is the QR analogue of solving the normal equations. Indeed,

$$R^T R = A^T A = XX^T + \lambda^2 I_m, \qquad R^T c = A^T b = Xy. \tag{27}$$

Thus QR and the normal equations define the same exact solution. Numerically, however, QR is preferable because it avoids explicitly forming and solving with $A^T A$, whose condition number is squared:

$$\kappa(A^T A) = \kappa(A)^2. \tag{28}$$

For small λ, this distinction is important, since Eq. (8) shows that $\kappa(A)$ already grows like $O(1/\lambda)$.

5.2 Algorithmic Structure

The QR-based solver can be summarized as follows:

Algorithm 3 QR Solver for the Regularized Least Squares Problem

Require: Data matrix $X \in \mathbb{R}^{m \times n}$; response $y \in \mathbb{R}^n$; parameter $\lambda > 0$

1: Form the linear operator $A = [X^T; \lambda I_m]$ and vector $b = [y; 0]$
2: Compute a thin QR factorization $A = Q_1 R$
3: Compute $c \leftarrow Q_1^T b$
4: Solve $Rw = c$ by back-substitution
5: **return** w

In practice, the matrix Q_1 is not formed explicitly. Forming it would require $O((n+m)m)$ storage and applying it as a dense matrix would obscure the sparse and structured nature of A. Instead, the orthogonal transformations used during the factorization are stored in compact form and later applied to b.

5.3 Cost for the Project Matrix

The matrix in this project is not a generic dense rectangular matrix. It has the block form

$$A = \begin{bmatrix} X^T \\ \lambda I_m \end{bmatrix}, \tag{29}$$

where X is tall-thin and the ML-CUP data dimension n is small compared with m. The lower block is diagonal, while all data-dependent information is contained in the n rows of X^T. A direct dense Householder factorization that ignores this structure would cost

$$O((n+m)m^2), \tag{30}$$

which contains a cubic term in m. This is not the right implementation model for the project requirement that the QR code should be at most quadratic in m.

The useful observation is that the row order is irrelevant for the least squares solution. If P is any permutation matrix, then

$$\|Aw - b\| = \|P(Aw - b)\|. \tag{31}$$

We may therefore work with the permuted system

$$\tilde{A} = \begin{bmatrix} \lambda I_m \\ X^T \end{bmatrix}, \qquad \tilde{b} = \begin{bmatrix} 0 \\ y \end{bmatrix}. \tag{32}$$

The first m rows already form the triangular factor $R_0 = \lambda I_m$. The n dense rows of X^T can then be inserted one at a time by triangularizing

$$\begin{bmatrix} R_{i-1} \\ x_i^T \end{bmatrix}, \qquad i = 1, \ldots, n, \tag{33}$$

where x_i denotes the i-th column of X, equivalently the i-th row of X^T. Each row insertion costs $O(m^2)$, so the total cost is

$$O(nm^2), \tag{34}$$

plus $O(m^2)$ for the final back-substitution and storage of the triangular factor. For the ML-CUP setting, where n is fixed and much smaller than m, this is quadratic in m. This is also consistent with the assignment constraint: the algorithm exploits the diagonal regularization block instead of treating λI_m as a dense matrix.

6 QR with Householder Reflectors

Householder reflectors are the standard stable mechanism for computing QR factorizations [5, 6]. A Householder reflector is an orthogonal matrix of the form

$$H = I - \tau u u^T, \qquad \tau = \frac{2}{u^T u}, \tag{35}$$

where $u \neq 0$. It is symmetric and orthogonal:

$$H^T = H, \qquad H^T H = I. \tag{36}$$

Given a vector $z \in \mathbb{R}^\ell$, the reflector is chosen so that all components below the first one are annihilated. Let

$$\alpha = -\operatorname{sign}(z_1) \|z\|_2, \qquad u = z - \alpha e_1. \tag{37}$$

Then

$$Hz = \alpha e_1. \tag{38}$$

Here sign(0) is taken as 1. The sign choice avoids cancellation when z_1 is already close to $\|z\|_2$, and is therefore important for numerical stability.

6.1 Dense Householder QR

For a generic dense matrix $A \in \mathbb{R}^{p \times m}$, Householder QR proceeds column by column. At step k, one constructs a reflector H_k acting only on rows $k, \ldots, p$ such that

$$H_k A_{k:p,k} = \begin{bmatrix} \alpha_k \\ 0 \\ \vdots \\ 0 \end{bmatrix}. \tag{39}$$

The same reflector is applied to the whole trailing submatrix $A_{k:p,k:m}$. After m steps,

$$H_m \cdots H_2 H_1 A = \begin{bmatrix} R \\ 0 \end{bmatrix}. \tag{40}$$

Since each reflector is orthogonal,

$$Q^T = H_m \cdots H_1, \qquad A = QR. \tag{41}$$

The important implementation point is that Q is never formed. Only the Householder vectors u_k and scalars τ_k are stored. To compute $Q^T b$, one applies the stored reflectors to b in the same order used during the factorization:

$$b \leftarrow H_k b = b - \tau_k u_k (u_k^T b). \tag{42}$$

This gives the vector $Q^T b$, whose first m entries define the right-hand side c in Eq. (26).

Algorithm 4 Compact Dense Householder QR

Require: Matrix $A \in \mathbb{R}^{p \times m}$ with $p \geq m$

for $k = 1, \ldots, m$ **do**
  $z \leftarrow A_{k:p,k}$
  $\alpha \leftarrow -\operatorname{sign}(z_1) \|z\|_2$
  $u \leftarrow z - \alpha e_1; \quad \tau \leftarrow 2/(u^T u)$
  $A_{k:p,k:m} \leftarrow A_{k:p,k:m} - \tau u (u^T A_{k:p,k:m})$
  Store (u, τ)
end for
$R \leftarrow \operatorname{triu}(A_{1:m,1:m})$
return R and stored reflectors

Algorithm 4 is the mathematically canonical version. It is useful for explaining QR, but for the project matrix it should be specialized as described next.

6.2 Structured Householder Row Insertion

Using the permuted augmented system (32), the QR factorization starts from the already triangular matrix $R_0 = \lambda I_m$. For each data row x_i^T and scalar right-hand side y_i, we maintain an upper triangular matrix R and a transformed right-hand side c. Inserting one row amounts to reducing

$$\begin{bmatrix} R \\ x_i^T \end{bmatrix} \tag{43}$$

back to triangular form. This can be done with m Householder reflectors, each with support on only two rows: the current triangular row and the active inserted row.

At position k, the only entry below the diagonal in column k is the active row entry a_k. The two-vector

$$z = \begin{bmatrix} R_{kk} \\ a_k \end{bmatrix} \tag{44}$$

is transformed by a two-dimensional Householder reflector

$$H_k^{(i)} = I_2 - \tau u u^T, \qquad H_k^{(i)} z = \begin{bmatrix} \rho \\ 0 \end{bmatrix}. \tag{45}$$

The same reflector is applied to the trailing entries of the two affected rows:

$$\begin{bmatrix} R_{k,k:m} \\ a_{k:m} \end{bmatrix} \leftarrow H_k^{(i)} \begin{bmatrix} R_{k,k:m} \\ a_{k:m} \end{bmatrix}, \qquad \begin{bmatrix} c_k \\ \eta \end{bmatrix} \leftarrow H_k^{(i)} \begin{bmatrix} c_k \\ \eta \end{bmatrix}, \tag{46}$$

where η is the active right-hand-side entry, initialized as $\eta = y_i$. After the m reflectors have been applied, the inserted row has been zeroed and can be discarded. The updated R and c are exactly what would be obtained by applying QR to all rows processed so far.

Algorithm 5 Structured Householder QR for $[X^T; \lambda I_m]$

Require: $X \in \mathbb{R}^{m\times n}$, $y \in \mathbb{R}^n$, $\lambda > 0$
$R \leftarrow \lambda I_m$; $\quad c \leftarrow 0 \in \mathbb{R}^m$
for $i = 1, \ldots, n$ **do**
  $a \leftarrow X_{:,i}$; $\quad \eta \leftarrow y_i$
  for $k = 1, \ldots, m$ **do**
    $z \leftarrow [R_{kk},\, a_k]^T$
    Build the Householder reflector H such that $Hz = [\rho,\, 0]^T$
    $[R_{k,k:m};\, a_{k:m}] \leftarrow H[R_{k,k:m};\, a_{k:m}]$
    $[c_k;\, \eta] \leftarrow H[c_k;\, \eta]$
    Store the reflector parameters if later products with Q or Q^T are required
  end for
end for
Solve $Rw = c$ by back-substitution
return w

6.3 Complexity and Numerical Properties

For each inserted row, the inner loop updates trailing row segments of lengths $m, m-1, \ldots, 1$. Therefore the cost per row is

$$\sum_{k=1}^{m} O(m-k+1) = O(m^2), \tag{47}$$

and processing all n rows of X^T costs $O(nm^2)$. The final triangular solve costs $O(m^2)$. The storage is $O(m^2)$ for the upper triangular factor R and $O(m)$ for the right-hand side c; if all reflectors are stored in order to apply Q and Q^T later, the additional storage is $O(nm)$ because each reflector has two-dimensional support.

The method remains backward stable because it is built entirely from orthogonal transformations. In contrast with the normal equations, it does not square the condition number. The regularization parameter λ also guarantees that the triangular factor is nonsingular, since

$$R^T R = A^T A = XX^T + \lambda^2 I_m \succ 0. \tag{48}$$

For very small λ, the problem is still more ill-conditioned, but the QR method is affected through $\kappa(A)$ rather than through $\kappa(A)^2$. This is the main reason for using QR as the direct baseline against which the L-BFGS iterations are compared.

7 Comparison: QR vs. L-BFGS

(To be completed after A2 implementation.)

A Proofs

Lemma 1 ($\hat{X}$ has full column rank)**.** *If $\lambda > 0$, then $\hat{X}^T\hat{X} = XX^T + \lambda^2 I_m \succ 0$.*

Proof. For any nonzero $v \in \mathbb{R}^m$: $v^T(XX^T + \lambda^2 I_m)v = \left\|X^T v\right\|^2 + \lambda^2 \|v\|^2 \geq \lambda^2 \|v\|^2 > 0$. □

Lemma 2 (Eigenvalue shift)**.** *If α is an eigenvalue of A, then $\alpha + c$ is an eigenvalue of $A + cI$.*

Proof. $Av = \alpha v \iff (A + cI)v = (\alpha + c)v$. □

Lemma 3 (Singular values of XX^T)**.** *The eigenvalues of $XX^T \in \mathbb{R}^{m\times m}$ are $\{\sigma_1^2, \ldots, \sigma_n^2, 0, \ldots, 0\}$, where $\sigma_1, \ldots, \sigma_n$ are the singular values of X. In particular, $\lambda_m(XX^T) = 0$ when $m > n$.*

Proof. Let $X = U\Sigma V^T$ be the SVD. Then $XX^T = U\Sigma\Sigma^T U^T$, where $\Sigma\Sigma^T = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_n^2, 0, \ldots, 0) \in \mathbb{R}^{m\times m}$. □

References

[1] J. Nocedal and S. J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.

[2] D. C. Liu and J. Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45(1):503–528, 1989.

[3] C. G. Broyden. The convergence of a class of double-rank minimization algorithms. *IMA Journal of Applied Mathematics*, 6(1):76–90, 1970.

[4] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*. Springer, 2nd edition, 2009.

[5] L. N. Trefethen and D. Bau III. *Numerical Linear Algebra*. SIAM, 1997.

[6] G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.

[7] A. Rodomanov and Y. Nesterov. New results on superlinear convergence of classical quasi-Newton methods. *Journal of Optimization Theory and Applications*, 188(3):744–769, 2021.

[8] Q. Jin and A. Mokhtari. Non-asymptotic superlinear convergence of standard quasi-Newton methods. *Mathematical Programming*, 194(1–2):435–468, 2022.

[9] P. Wolfe. Convergence conditions for ascent methods. *SIAM Review*, 11(2):226–235, 1969.